Data Structures
Trees



Thareendhra Keerthi Wijayasiriwardhane

Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk



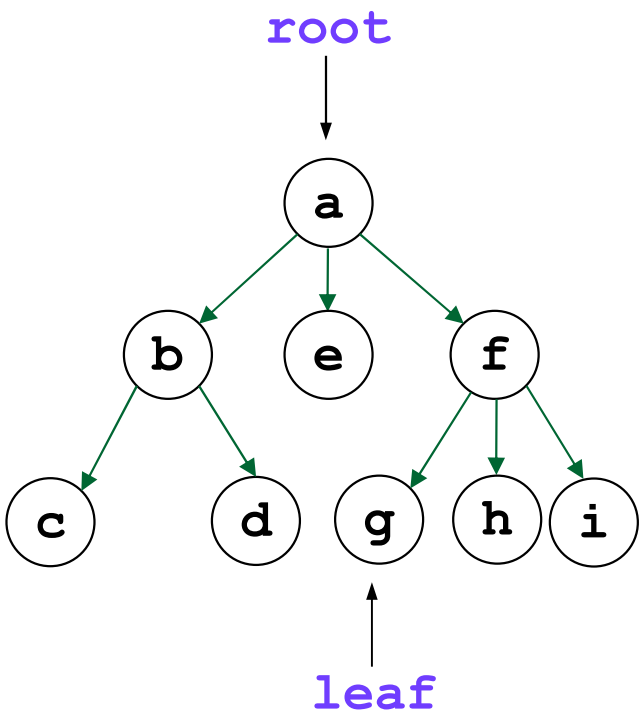


Learning Outcomes

- On completion of this session, you should be able to
  - Define tree data structure
  - Identify the root, leaves, subtrees and order of a tree
  - Traverse a tree using preorder, inorder and postorder traversal
  - Declare array-based and linked list-based trees
  - Implement array-based and linked list-based trees
  - Use trees to solve various real-world problems

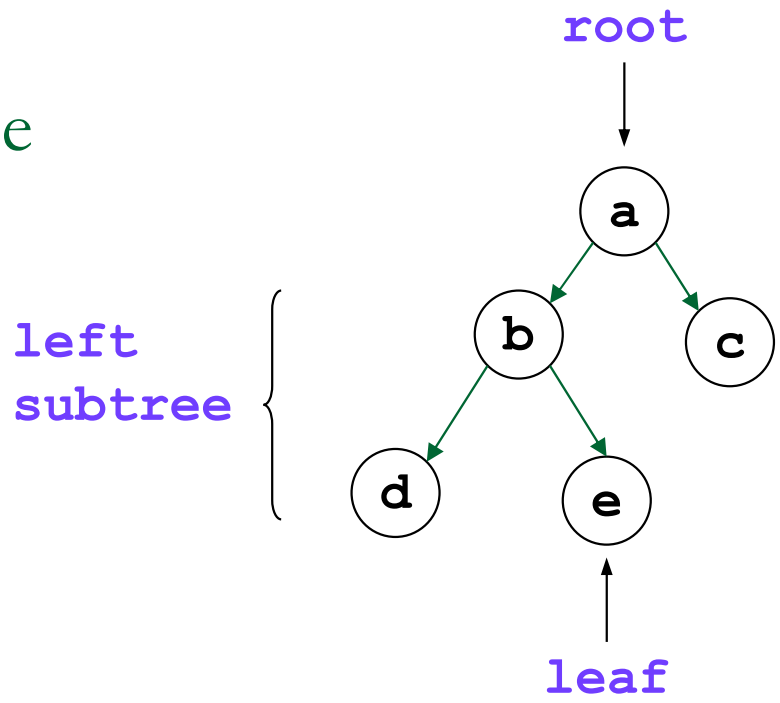
Tree

- A set of nodes that holds atomic or composite data
- Nodes are connected to other nodes with edges
- Root
- Leaf
- Order



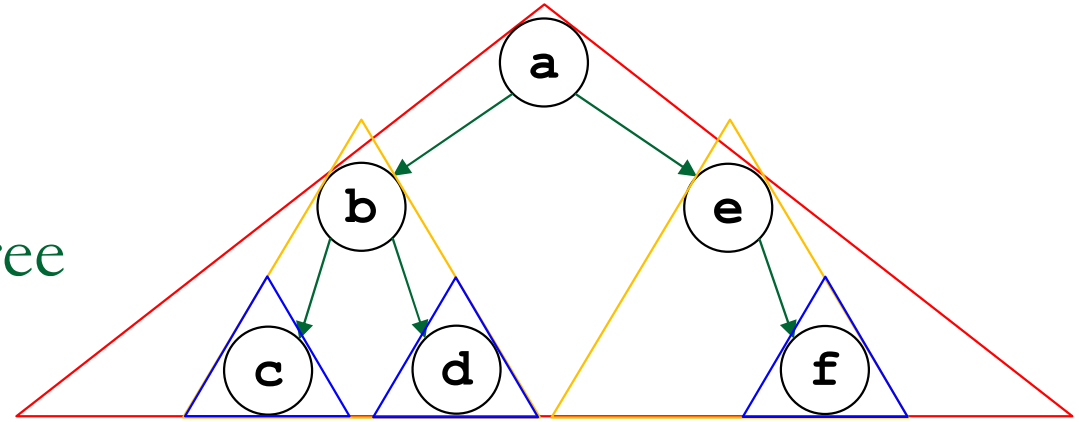
Binary Tree

- A tree with an order of two
- A finite set of elements partitioned into three subsets
  - Root
  - Left subtree
  - Right subtree



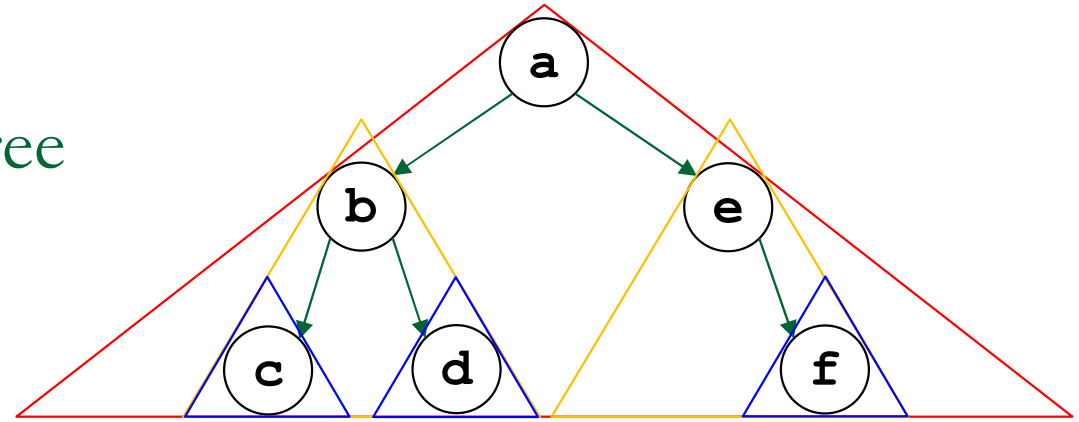
Binary Tree Traversal

- Visiting each node of a binary tree once and only once in a predetermined sequence
- Preorder traversal (each node is visited when it is traversed)
  - Visit the root
  - Traverse the left subtree in preorder
  - Traverse the right subtree in preorder



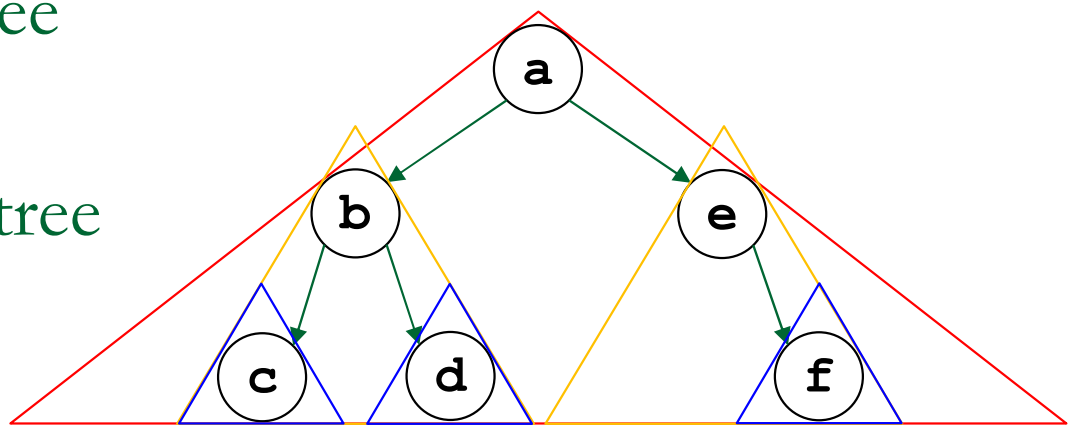
Binary Tree Traversal

- Inorder traversal (each node is not visited until its left subtree is traversed)
  - Traverse the left subtree in inorder
  - Visit the root
  - Traverse the right subtree in inorder



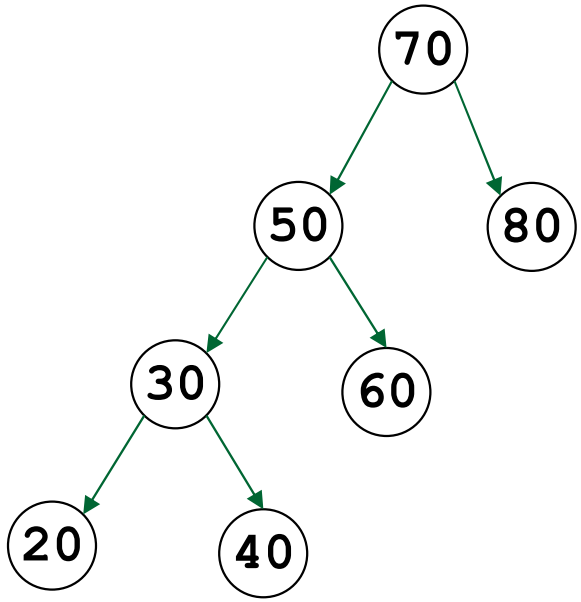
Binary Tree Traversal

- Postorder traversal (each node is not visited until both of its subtrees are traversed)
  - Traverse the left subtree in postorder
  - Traverse the right subtree in postorder
  - Visit the root



Binary Search Tree

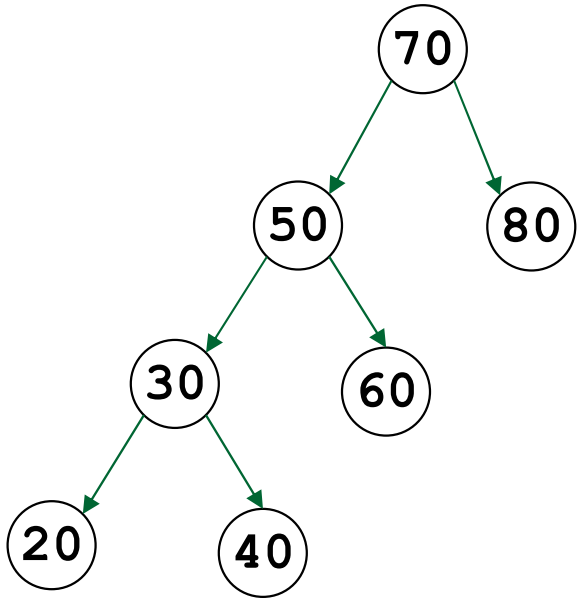
- A binary tree in which
  - Data in any node of the left subtree is less than that of the root,
  - Data in any node of the right subtree is greater than or equal to that of the root and
  - Each subtree is itself a binary search tree



Sorting Algorithm

```
IF tree is empty
   create a node and insert incoming data item
ELSE
   label
   IF incoming item < item stored in current node
      visit left child node of current node
      IF not successful
         insert incoming item as left child node
      ELSE
         go to label
      END IF
   ELSE
      visit right child node of current node
      IF not successful
         insert incoming item as right child node
      ELSE
         go to label
      END IF
   END IF
END IF
traverse tree in inorder
```

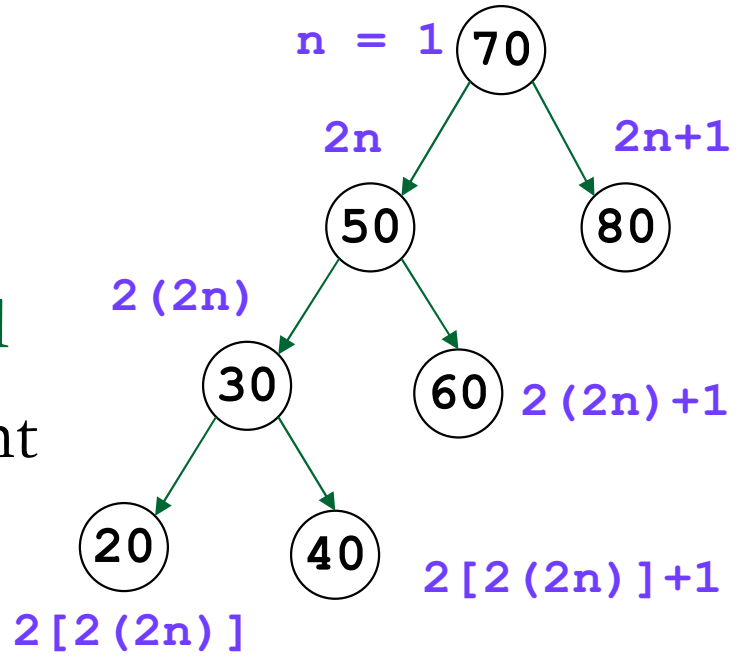
incoming item 60



20, 30, 40, 50, 60, 70, 80

Array-based Tree

- Conceptual Tree
- Physical Tree
  - Root of the tree is at array index 1
  - Node in array index n is the parent of child nodes 2n and 2n + 1
    - Left child node is at index 2n
    - Right child node is at index 2n + 1



0	1	2	3	4	5	6	7	8	9	10	11
	70	50	80	30	60			20	40		

Array-based Tree

- Global Variables

```
p POINTER to DATATYPE, size INTEGER
```

- create Operation

```
ALGORITHM create(VAL s INTEGER)
1  size ← s
2  p ← ALLOCATE MEMORY DATATYPE[size]
3  i ← 0
4  LOOP (i < size)
5     p[i] ← 0
6     i ← i + 1
7  END LOOP
END create
```

- destroy Operation

```
ALGORITHM destroy()
1  RELEASE MEMORY[] p
END destroy
```

Array-based Tree

- insert Operation

```
ALGORITHM insert(VAL d DATATYPE, VAL n INTEGER ← 1)
1  IF (n < size)
2     IF (p[n] = 0)
3        p[n] ← d
4        RETURN true
5     ELSE
6        IF (d < p[n])
7           RETURN insert(d, 2n)
8        ELSE
9           RETURN insert(d, 2n + 1)
10       END IF
11    END IF
12 ELSE
13    RETURN false
14 END IF
END insert
```

Array-based Tree

- traversePreorder Operation

```
ALGORITHM traversePreorder(VAL n INTEGER ← 1)
1  IF (n < size)
2     IF (p[n] ≠ 0)
3        PRINT(p[n])
4        traversePreorder(2n)
5        traversePreorder(2n + 1)
6     END IF
7  END IF
END traversePreorder
```

Array-based Tree

- traverseInorder Operation

```
ALGORITHM traverseInorder(VAL n INTEGER ← 1)
1  IF (n < size)
2     IF (p[n] ≠ 0)
3        traverseInorder(2n)
4        PRINT(p[n])
5        traverseInorder(2n + 1)
6     END IF
7  END IF
END traverseInorder
```

Array-based Tree

- traversePostorder Operation

```
ALGORITHM traversePostorder(VAL n INTEGER ← 1)
1  IF (n < size)
2     IF (p[n] ≠ 0)
3        traversePostorder(2n)
4        traversePostorder(2n + 1)
5        PRINT(p[n])
6     END IF
7  END IF
END traversePostorder
```

Array-based Tree

- test Algorithm

```
ALGORITHM test()
1  create(treeSize)
2  READ(data)
3  LOOP(data ≠ 0 AND insert(data))
4     READ(data)
5  END LOOP
6  traverseInorder()
7  destroy()
END test
```

Array-based Tree Implementation

```
//ArrayTree.h header file
#include <iostream>
class tree
{
   public:
      tree(int); //create() operation
      ~tree(); //destroy() operation
      int insert(int, int = 1);
      void traversePreorder(int = 1);
      void traverseInorder(int = 1);
      void traversePostorder(int = 1);
   private:
      int size;
      int *p;
};
```

Array-based Tree Implementation

```
tree::tree(int s)
{
   size = s;
   p = new int[size]; //allocates memory
   for(int i = 0; i < size; i++)
      p[i] = 0;
}
tree::~tree()
{
   delete[] p; //releases memory
}
```

Array-based Tree Implementation

```
int tree::insert(int d, int n)
{
   if (n < size)
      if (p[n] == 0)
      {
         p[n] = d;
         return 1;
      }
      else
         if (d < p[n])
            return insert(d, 2 * n);
         else
            return insert(d, 2 * n + 1);
   else
      return 0;
}
```

Array-based Tree Implementation

```
void tree::traversePreorder(int n)
{
   if (n < size)
      if (p[n] != 0)
      {
         std::cout << p[n] << endl;
         traversePreorder(2 * n);
         traversePreorder(2 * n + 1);
      }
}
```

Array-based Tree Implementation

```
void tree::traverseInorder(int n)
{
   if (n < size)
      if (p[n] != 0)
      {
         traverseInorder(2 * n);
         std::cout << p[n] << endl;
         traverseInorder(2 * n + 1);
      }
}
```

Array-based Tree Implementation

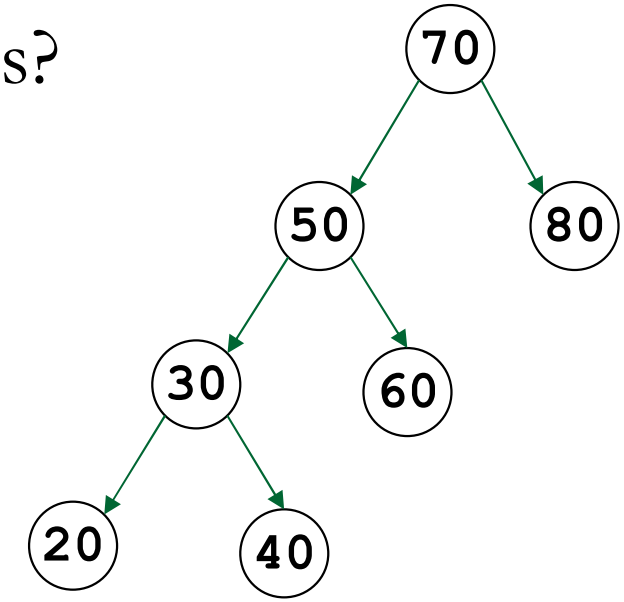
```
void tree::traversePostorder(int n)
{
   if (n < size)
      if (p[n] != 0)
      {
         traversePostorder(2 * n);
         traversePostorder(2 * n + 1);
         std::cout << p[n] << endl;
      }
}
```

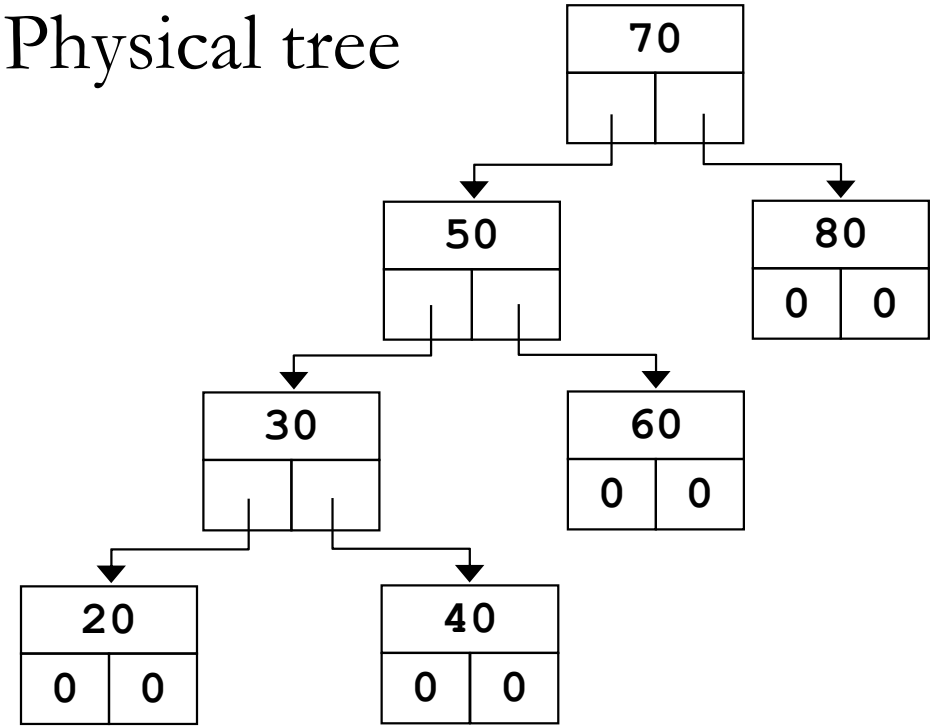
Array-based Tree Implementation

```
//TestArrayTree.cpp source file
#include <iostream>
#include "ArrayTree.h"
int main()
{
   const int treeSize = 10;
   tree t(treeSize);
   int data;
   std::cin >> data;
   while(data != 0 && t.insert(data))
      std::cin >> data;
   std::cout << endl;
   t.traverseInorder();
   return 0;
}
```

Linked List-based Tree

- Problems with Array-based Trees?
- Conceptual tree
- Physical tree





Linked List-based Tree

- Global Variables

```
TYPE node
    data DATATYPE
    left POINTER to node
    right POINTER to node
END node
root POINTER to node
```

- create Operation

```
ALGORITHM create()
1  root ← null
END create
```

- destroy Operation

```
ALGORITHM destroy()
1  removeNode(root)
END destroy
```

Linked List-based Tree

- removeNode Operation

```
ALGORITHM removeNode(VAL p POINTER to node)
1  IF (p ≠ null)
2     removeNode(p->left)
3     removeNode(p->right)
4     RELEASE MEMORY p
5  END IF
END removeNode
```

- insert Operation

```
ALGORITHM insert(VAL d DATATYPE)
1  addNode(root, d)
END insert
```

Linked List-based Tree

- addNode Operation

```
ALGORITHM addNode(REF p POINTER to node, VAL d DATATYPE)
1  IF (p = null)
2     p ← ALLOCATE MEMORY node
3     p->data ← d
4     p->left ← null
5     p->right ← null
6  ELSE
7     IF (d < p->data)
8        addNode(p->left, d)
9     ELSE
10       addNode(p->right, d)
11    END IF
12 END IF
END addNode
```

Linked List-based Tree

- traverse Operation

```
ALGORITHM traverse(VAL n INTEGER)
1  IF (n = 1)
2     traversePreorder(root)
3  ELSE
4     IF (n = 2)
5        traverseInorder(root)
6     ELSE
7        IF (n = 3)
8           traversePostorder(root)
9        END IF
10    END IF
11 END IF
END traverse
```

Linked List-based Tree

- traversePreorder Operation

```
ALGORITHM traversePreorder(VAL p POINTER to node)
1  IF (p ≠ null)
2     PRINT(p->data)
3     traversePreorder(p->left)
4     traversePreorder(p->right)
5  END IF
END traversePreorder
```

Linked List-based Tree

- traverseInorder Operation

```
ALGORITHM traverseInorder(VAL p POINTER to node)
1  IF (p ≠ null)
2     traverseInorder(p->left)
3     PRINT(p->data)
4     traverseInorder(p->right)
5  END IF
END traverseInorder
```

Linked List-based Tree

- traversePostorder Operation

```
ALGORITHM traversePostorder(VAL p POINTER to node)
1  IF (p ≠ null)
2     traversePostorder(p->left)
3     traversePostorder(p->right)
4     PRINT(p->data)
5  END IF
END traversePostorder
```

Linked List-based Tree

- test Algorithm

```
ALGORITHM test()
1  create()
2  READ(data)
3  LOOP(data ≠ 0)
4      insert(data)
5      READ(data)
6  END LOOP
7  traverse(2)
8  destroy()
END Test
```

Linked List-based Tree Implementation

```
//LinkedListTree.h file
#include <iostream>
struct node
{
   int data;
   node *left;
   node *right;
};
```

```
class tree
{
   public:
      tree(); //create() operation
      ~tree(); //destroy() operation
      void insert(int);
      void traverse(int);
   private:
      void removeNode(node *);
      void addNode(node *&, int);
      void traversePreorder(node *);
      void traverseInorder(node *);
      void traversePostorder(node *);
      node *root;
};
```

Linked List-based Tree Implementation

```
tree::tree()
{
   root = 0; //sets to null
}
tree::~tree()
{
   removeNode(root);
}
void tree::removeNode(node *p)
{
   if (p)
   {
      removeNode(p->left);
      removeNode(p->right);
      delete p;
   }
}
```

Linked List-based Tree Implementation

```
void tree::insert(int d)
{
   addNode(root, d);
}
```

```
void tree::addNode(node *&p, int d)
{
   if (p == 0)
   {
      p = new node;
      p->data = d;
      p->left = 0;
      p->right = 0;
   }
   else
      if (d< p->data)
         addNode(p->left, d);
      else
         addNode(p->right, d);
}
```

Linked List-based Tree Implementation

```
void tree::traverse(int n)
{
   switch(n)
   {
      case 1:
         traversePreorder(root);
         break;
      case 2:
         traverseInorder(root);
         break;
      case 3:
         traversePostorder(root);
         break;
   }
}
```

Linked List-based Tree Implementation

```
void tree::traversePreorder(node *p)
{
   if (p != 0)
   {
      std::cout << p->data << endl;
      traversePreorder(p->left);
      traversePreorder(p->right);
   }
}
```

Linked List-based Tree Implementation

```
void tree::traverseInorder(node *p)
{
   if (p != 0)
   {
      traverseInorder(p->left);
      std::cout << p->data << endl;
      traverseInorder(p->right);
   }
}
```

Linked List-based Tree Implementation

```
void tree::traversePostorder(node *p)
{
   if (p != 0)
   {
      traversePostorder(p->left);
      traversePostorder(p->right);
      std::cout << p->data << endl;
   }
}
```

Linked List-based Tree Implementation

```
//TestLinkedListTree.cpp source file
#include <iostream>
#include "LinkedListTree.h"
int main()
{
   tree t;
   int data;
   std::cin >> data;
   while(data != 0)
   {
      t.insert(data);
      std::cin >> data;
   }
   std::cout << endl;
   t.traverse(2);
   return 0;
}
```

Tree Applications

- Represent hierarchically related data
- Represent mathematical expressions
- Support data compression and encoding
- Efficiently search larger dynamic sets of data
- Recommended References
  - *Data Structures: A Pseudo code Approach with C++*, Gilberg and Forouzan, Chapter 7, pp. 308 – 310, 322 – 324, 327 – 330, 339 – 352

Questions